SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 1 – APPLICATION BASED QUESTIONS

Course Code:	CSA06	**Course Title:**	Design and Analysis of Algorithms
CO Assessed:	CO1 – Precision (BL1, BL2, BL3)	**Assessment:**	Assessment Tool 1 – Application Based Questions
Weightage:	40% of CIA	**Total Marks:**	100 (2 Questions × 50 Marks)
CO1 Statement:	*Determine algorithm efficiency using asymptotic notations and mathematical techniques including Master Theorem and substitution method. (BL1, BL2, BL3)*		
Student Name:	Aryan Arun	**Reg. No.:**	192411157
Section / Batch:	BE CSE – 3rd Year	**Date:**	09/07/2026

Instructions to Students

- Answer BOTH questions. Each question carries 50 Marks. Total: 100 Marks.
- Clearly show all steps in derivations and complexity analysis.
- Use proper asymptotic notation (Big-O, Big-Θ, Big-Ω) wherever required.
- Justify each step in recurrence solving using appropriate methods.
- Neat presentation and logical explanation are mandatory

Question Type	Focus Area	Questions	Marks
Application-Based Questions	Apply Master Theorem and asymptotic analysis to real-world scenarios	Q1 – Q2	Q1 –50 Q2 –50
	GRAND TOTAL:	**100 Marks**	

SECTION A – APPLICATION BASED QUESTIONS

Code of Conduct

I ________Aryan Arun (192411157)__________________ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: ______Aryan Arun___________________

RUBRICS FOR CALCULATION

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Problem Context	20%	Clearly understands the problem and accurately identifies all requirements	Good understanding with minor gaps	Basic understanding; some requirements unclear	Poor understanding or misinterpretation of the problem
Application of Concepts	30%	Accurately applies appropriate concepts to solve complex problems	Applies relevant concepts correctly with minor errors	Applies basic concepts with limited accuracy	Fails to apply appropriate concepts
Problem-Solving Approach	20%	Logical, well-structured, and efficient approach	Mostly logical approach with minor gaps	Basic approach with some inconsistencies	Illogical or unclear approach
Accuracy of Solution	20%	Completely correct solution with proper justification	Mostly correct with minor errors	Partially correct solution	Incorrect or incomplete solution
Clarity	10%	Clear, well-organized, and properly explained solution	Understandable with minor clarity issues	Limited clarity and explanation	Poorly presented and difficult to understand

Marks Evaluation:

Question No.	Understanding of Problem Context (20%)	Application of Concepts (30%)	Problem-Solving Approach (20%)	Accuracy of Solution (20%)	Clarity (10%)	Total Marks
Q1 (50 Marks)						
Q2 (50 Marks)						
					Overall Total (100)	

Faculty In-charge	Course Coordinator	Head of Department
Signature: ________________	Signature: ________________	Signature: ________________
Date: ____________________	Date: ____________________	Date: ____________________

CSA0624 Design And Analysis Of Algorithms

NAME: ARYAN ARUN

REG.NO: 192411157

CO1_AT1

1. Application: Online Video Streaming Platform

Given:

A video streaming platform processes user data using the recurrence relation:

$$T(n) = 2T(n/2) + n$$

Where: $a = 2$ $f(n) = n$

$b = 2$

Step 1: Calculate $n^{\log_b a}$

$$n^{\log_2 2} = n$$

Thus,

$$f(n) = n = \Theta(n^{\log_b a})$$

it satisfies Case 2 for Master's Theorem

Therefore: $T(n) = \Theta(n \log n)$

Time Complexity:

* Best Case: $\Omega(n \log n)$
* Average Case: $\Theta(n \log n)$

* Worst Case: $O(n \log n)$

Scalability Analysis

As the number of users increases, the processing time grows at the rate of $n \log n$, which is efficient for large scale streaming platforms. The algorithm scales well because the data is divided into equal halves at every recursive step.

Conclusion:

The recurrence satisfies case 2 of Master's theorem

$$T(n) = \Theta(n \log n)$$

2. Application: Social Media Feed Processing

Given:

$$T(n) = T(n-1) + n$$

Step 1: Expand

$$T(n) = T(n-2) + (n-1) + n$$

$$T(n) = T(n-3) + (n-2) + (n-1) + n$$

Continuing:

$$T(n) = T(1) + 2 + 3 + \dots + n$$

Step 2: Sum of natural numbers

$$1 + 2 + 3 + \dots + n = \frac{n(n+1)}{2}$$

Hence,

$$T(n) = T(1) + \frac{n(n+1)}{2}$$

Ignoring Constants

$$T(n) = \Theta(n^2)$$

Complexity:

* Best Case: $\Omega(n^2)$
* Average Case: $\Theta(n^2)$
* Worst Case: $O(n^2)$

Performance Analysis:

As the number of social media posts increases, the running time increases quadratically. This leads to slower processing for very large datasets.

Conclusion:

The substitution method gives

$$T(n) = \Theta(n^2)$$

3. Application: E-Commerce Recommendation System

The system uses two algorithms

* Algorithm 1 → $O(n \log n)$
* Algorithm 2 → $O(n^2)$

Comparison:

Complexity	Performance
O nlogn	Faster
$O(n^2)$	Slower

Growth Rate Analysis:

For small datasets, both algorithms perform reasonably well.
However, as the input size increases:

* O(nlogn) grows slowly
* $O(n^2)$ grows rapidly

For example:

When $n = 1000$

$n\log n \approx 10000$

Whereas

$n^2 = 1000000$

Thus the quadratic operation performs nearly 100 times more operations.

Scalability:

The O(nlogn) algorithm scales efficiently and is suitable for large recommendation systems.
The $O(n^2)$ algorithm becomes inefficient for millions of users

4. Application: Cloud Storage Processing System

Given, $T(n) = 3T(n/2) + n$

Parameters : * $a = 3$, * $b = 2$, * $f(n) = n$

Step1: Compute ~~n^{lo}~~ $n^{\log_2 3}$

Since $\log_2 3 \approx 1.585$

Therefore, $n^{\log_2 3} = n^{1.585}$

Step 2: Compare $f(n) = n$ with $n^{1.585}$

Since $n < n^{1.585}$

it satisfies Case 1 of Master's theorem.

Time Complexity:

$$T(n) = \Theta(n^{\log_2 3})$$

approximately: $\Theta(n^{1.585})$

Scalability:

The algorithm performs better than quadratic algorithms but is slower than $n \log n$ algorithms.

It remains suitable for large cloud storage systems because the data is divided recursively.

Conclusion:

The recurrence satisfies Case 1. Final Complexity: $\Theta(n^{1.585})$

5. Application: Financial Analytics System

Given,

$$T(n) = 2T(n/2) + n\log n$$

Parameters: * $a = 2$, * $b = 2$, * $f(n) = n\log n$

Step 1:

Calculate

$$n^{\log_2 2} = n$$

Now Compare

$$f(n) = n\log n$$

with

$$n^{\log_b a} = n$$

Since $f(n) = \Theta(n\log n)$ which is $\Theta(n^{\log_b a} \log^1 n)$

This satisfies the extended Case 2 of the Master Theorem.

Time Complexity: $T(n) = \Theta(n\log^2 n)$

Performance Analysis:

The Algorithm performs efficiently for large financial datasets. Although the extra $\log n$ factor increases computation, the overall growth is still much slower than quadratic time.

Conclusion:

The recurrence satisfies the extended Case 2 of the Master Theorem.

Final Complexity: $\Theta(n\log^2 n)$